AIORI

ADVANCED INTERNET OPERATIONS RESEARCH IN INDIA
AIORI Hackathon 3 • National Technical Challenge

Track: IPv6 Deployment & Tooling

A3-PS027-TC001

IPv6 Website & Dependency Readiness Auditor

A Full-Stack Diagnostic Engine (V6Lighthouse) Detecting Client-Side Failures Caused by Legacy IPv4 Dependencies on Pure-IPv6 Networks.

Saurabh Gangwar
s73771934@gmail.com
Invertis University • Computer Science & Engineering

AIORI Hackathon 3
October 5, 2026

3 Problem Description

Clear outline of the problem being addressed. Key operational issues & failure modes.

The Partial Migration Trap

Enterprises configure AAAA DNS records on their apex servers, assuming 100% IPv6 readiness. However, modern frontends dynamically link dozens of external assets.

Unverified 3rd-Party Dependencies

Modern web apps depend on third-party CDNs, fonts, analytics, and payment SDKs (e.g. Stripe, jQuery, Google Fonts) hosted on unvetted, legacy IPv4-only domains.

Silent Pure-IPv6 Network Failures

Clients on pure-IPv6 cellular networks or enterprise subnets fail to route to IPv4 assets, causing missing UI styling, dropped checkouts, and stalled JavaScript scripts.

Diagnostic Tooling Vacuum

Standard tools (Ping, Dig, Google Lighthouse) evaluate only the apex hostname, offering zero visibility into the third-party dependency graph during runtime.

3 Background

Context necessary to understand the transition gap and the proposed optimization.

Global IPv6 Transition

IPv4 address exhaustion has driven global mobile operators and tier-1 service providers to deploy pure-IPv6 access networks utilizing transition mechanisms (NAT64/DNS64).

The Dual-Stack Fallacy

Websites believe dual-stack apex deployment solves compatibility. In reality, client browsers execute external script tags that bypass Happy Eyeballs fallback algorithms entirely.

Proposed Optimization

PS-027 bridges this gap: spawning headless browser sandboxes to harvest 100% of runtime outgoing HTTP requests, verifying each domain's IPv6 reachability via parallel low-level DNS.

3

Literature & RFC Review

Detailed study of existing internet standards, specifications, and academic proposals.

RFC 8200: Internet Protocol Version 6 (IPv6)

Base Standard

Defines the 128-bit address architecture, extension header processing, and mandatory DNS AAAA record mapping for internet nodes and hosts.

RFC 6555: Happy Eyeballs v2 Architecture

Dual-Stack Fallback

Specifies dual-stack connection racing to reduce user-visible delays. Documents how pure-IPv6 client environments fail when third-party servers provide no AAAA endpoints.

RFC 6883: IPv6 Guidance for Web & App Providers

Operational Guidance

Explicitly warns enterprise content providers that partial dual-stack transitions fail when dependent assets (fonts, APIs, analytics) are hosted on legacy infrastructure.

W3C Resource Timing & Puppeteer Interception

Runtime Spec

Specifies browser-level network timing and interception hooks (PerformanceResourceTiming), enabling accurate isolation of external third-party origins.

3

Proposed Solution

Detailed description of the V6Lighthouse full-stack auditing platform.

STEP 01

URL Ingestion

Developer inputs target hostname. Express orchestrator sanitizes schema, enforces protocol checks, and initializes job.

STEP 02

Apex DNS Audit

Native resolver validates primary server infrastructure: checking AAAA, NS, and MX records in parallel.

STEP 03

Headless Harvester

Puppeteer launches headlessly, intercepts outgoing HTTP requests, isolates external origins, and tags MIME types.

STEP 04

Scoring & Remediation

Computes weighted Readiness Score (0–100%) and generates context-aware CDN migration and local hosting recommendations.

Key Architectural Advantage: Decoupled low-level DNS resolution completely avoids browser-internal cache spoofing. **100% Deterministic**

3 Optimization Proposed

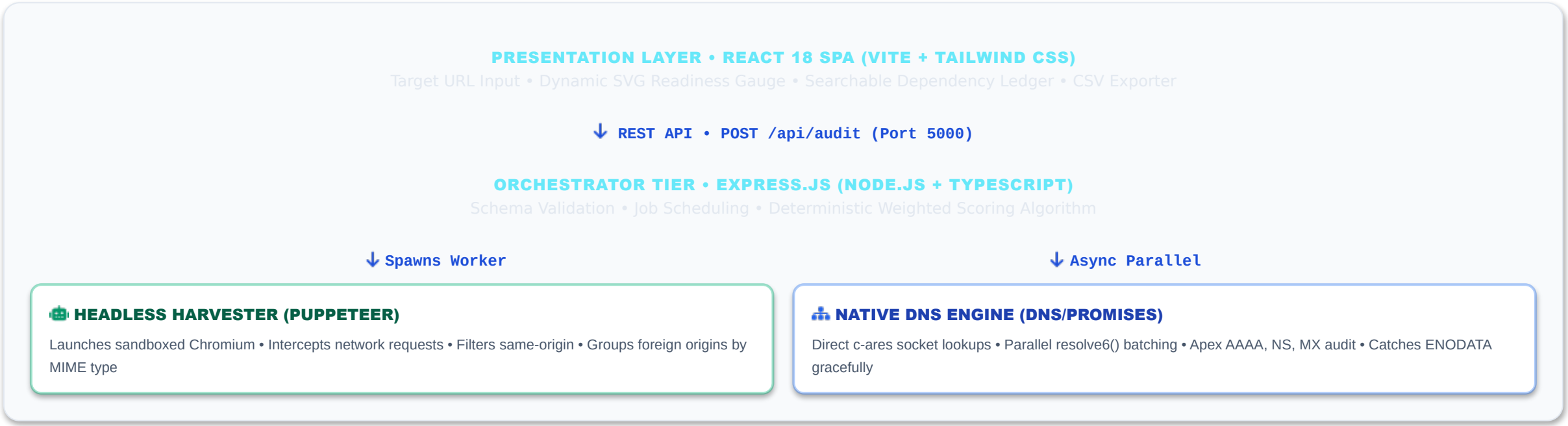
Technical comparison between traditional tools and the proposed V6Lighthouse auditor.

Capability Dimension	Traditional IPv6 Scanners	Proposed V6Lighthouse Auditor
Audit Scope	Apex domain only (AAAA lookup)	**100% of runtime external dependencies (JS, CSS, fonts, APIs)**
Harvesting Method	Static HTML regex parsing	**Dynamic headless Puppeteer network request interception**
DNS Accuracy	Browser cache pollution risks	**Parallel native c-ares low-level asynchronous resolver**
Output & Guidance	Binary Pass/Fail indicator	**Lighthouse-style weighted score + targeted CDN fixes + CSV export**
CI/CD Gate Readiness	Manual web console checks	**Headless CLI / JSON exit codes for GitHub Actions pipelines**

3

Solution Architecture & Design

Visual: Decoupled subsystem architecture & interaction model.



3

Data Flow Diagram

Sequential data pipeline: From URL ingestion to automated remediation.

STAGE 1

Target Ingestion

User enters URL; Express validates protocol and hostname.

`POST /api/audit`

STAGE 2

Apex DNS Check

Parallel DNS lookup for AAAA, NS, and MX server records.

`resolve6(host)`

STAGE 3

DOM Harvesting

Puppeteer loads DOM, intercepts requests, and maps external CDNs.

`page.on('request')`

STAGE 4

Parallel Checks

Batch verification of all captured foreign dependency domains.

`Promise.all DNS`

STAGE 5

Score & Export

Computes 0-100% score; outputs interactive table & CSV.

`JSON • CSV Blob`

Average Execution Time

8–12 seconds to fully render DOM and audit 50+ external dependencies.

Deduplication Gate

Maps 200+ raw network calls into unique, actionable third-party origin hosts.

Zero Cache Interference

Forces authoritative upstream queries to eliminate false positive readiness.

3

Deployment Timeline

Milestones and methodology for engineering execution and platform rollout.

PHASE 1 — Week 1

Core Engine & DNS

Initialize Express TypeScript monorepo, configure Puppeteer request interception, and implement parallel native DNS resolvers.

PHASE 2 — Week 2

UI & Scoring Engine

Build React 18 SPA dashboard, integrate circular health gauges, build dependency data tables, and implement CSV downloader.

PHASE 3 — Week 3

Advanced Differentiators

Build simulated pure-IPv6 sandbox routing tests and implement automated CDN code rewriter for instant script remediation.

PHASE 4 — Week 4

CI/CD & Scale Out

Package headless CLI for GitHub Actions gating (--fail-under 80) and deploy multi-region test nodes.

Bibliography I

Authoritative standards, RFC specifications, and academic references.

[1] **S. Deering, R. Hinden.** "Internet Protocol, Version 6 (IPv6) Specification." *IETF RFC 8200 (Internet Standard), July 2017.*

[2] **D. Wing, A. Yourtchenko.** "Happy Eyeballs: Success with Dual-Stack Hosts." *IETF RFC 6555, Proposed Standard, April 2012.*

[3] **M. Blanchet.** "IPv6 Guidance for Internet Content Providers and Application Service Providers." *IETF RFC 6883, Informational, March 2013.*

[4] **W3C Web Performance Working Group.** "Resource Timing Level 2." *W3C Recommendation, World Wide Web Consortium, 2019.*

[5] **Chrome DevTools Team.** "Puppeteer: Headless Chrome Node.js API for Network Interception." *Google Open Source Documentation, 2024.*



Thank You for Your Attention!

Open for Questions, Technical Discussion & Live Audit Demonstrations

Problem Statement: **PS-027 (A3-PS027-TC001)**

Project Name: **V6Lighthouse Auditor**

Presenter: **Saurabh Gangwar**

Institution: **Invertis University**